

City Tour Package & Ride Management System

8-Member Team Task Assignment

Beginner-friendly, console-based OOP project using arrays for in-memory storage. No database and no enum/enumeration.

1. Team Responsibilities

Member	Module	Main Classes	Main Responsibilities
1	Authentication & Users	User, Customer, Rider, Admin, Authenticatable	Registration, login/logout, role-based authentication, profiles, user IDs and validation.
2	City & Package Management	City, TourPackage	Cities, tour packages, categories, base price, duration, maximum persons and package status.
3	Destinations & Package Sections	Destination, PackageSection	Destinations, package checkpoints, section numbering, completion tracking and completion time.
4	Vehicle & Dynamic Pricing	Vehicle, Bike, Auto, Car, PricingCalculator	Vehicle selection, capacity, availability, passenger charges, vehicle charges and dynamic final price.
5	Customer Booking	Booking	Package/person/vehicle selection, booking confirmation, booking ID, date/time, status, history and cancellation.
6	Rider & Ride Management	Ride, Rider operations	Assigned rides, accept/reject, reached, pickup, start, section completion, ride completion and ride history.
7	Earnings, Reports & Exceptions	Earning, Custom Exceptions	Earnings, ride duration, admin reports, validation and exception handling.
8	Main Console & Integration	Main, SystemData	Main menus, arrays, sample data, module integration and full-system testing.

2. OOP Concept Distribution

Member	Main OOP Concepts	Viva Focus
1	Abstraction, Interface, Inheritance, Encapsulation	Abstract User, Authenticatable interface and role subclasses.
2	Encapsulation, Aggregation, Arrays	City contains multiple TourPackage objects.
3	Composition, Association, Date/Time	TourPackage contains PackageSection objects and tracks checkpoint completion.
4	Abstraction, Inheritance, Polymorphism	Bike, Auto and Car override vehicle charge calculation.
5	Association, Encapsulation, Date/Time	Customer creates and manages Booking objects.
6	Association, State Management, Date/Time	Ride status progression and package section completion.
7	Exception Handling, Date/Time	Custom exceptions, earnings and ride duration.
8	Arrays, Static/Non-static, Integration	SystemData storage and Main controller.

3. Development Order

- Phase 1: Member 1 — Users & Authentication
- Phase 2: Members 2 + 3 — Cities, Packages, Destinations & Sections
- Phase 3: Member 4 — Vehicles & Pricing
- Phase 4: Member 5 — Customer Booking
- Phase 5: Member 6 — Rider & Ride
- Phase 6: Member 7 — Earnings, Reports & Exceptions
- Phase 7: Member 8 — Main Console, SystemData, Integration & Testing

4. Important Project Rules

- Use arrays only for temporary in-memory storage. No database.
- Do not use enum/enumeration. Use String status values with shared static constants.
- Ride status flow: ASSIGNED → ACCEPTED → REACHED → PICKED_UP → IN_PROGRESS → COMPLETED.
- A rider cannot complete the ride until every package section is completed.

- $\text{Dynamic price} = \text{Base Package Price} + \text{Passenger Charge} + \text{Vehicle Charge} + \text{Additional Charges}$.
- All members should agree on class names, fields, methods and relationships before integration.
- Member 8 integrates the modules but does not replace the work of the other seven members.

5. Core Class Relationships

```
User (abstract)
|
+-- Customer
|
+-- Rider
|
+-- Admin

Authenticatable (interface)
|
+-- Customer
+-- Rider
+-- Admin

Vehicle (abstract)
|
+-- Bike
+-- Auto
+-- Car

City 1 ----- * TourPackage

TourPackage 1 ---- * PackageSection

PackageSection ---- Destination

Customer 1 ----- * Booking

Booking ---- TourPackage
Booking ---- Vehicle
Booking ---- Rider

Booking 1 ----- 0..1 Ride

Ride ---- PackageSection

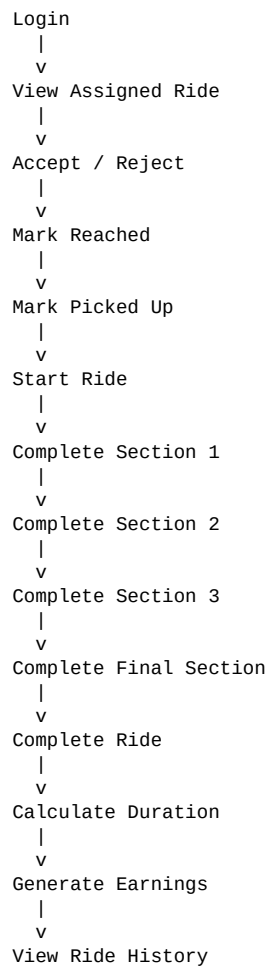
Ride ---- Earning

Rider ---- Earning
```

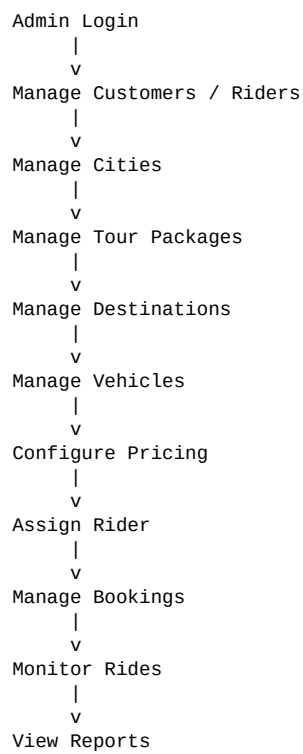
6. Customer Flow

```
Register / Login
|
v
View Cities
|
v
View Tour Packages
|
v
View Destinations
|
v
Select Number of Persons
|
v
Select Vehicle
|
v
Calculate Dynamic Price
|
v
Confirm Booking
|
v
Rider Assigned
|
v
View Current Ride
|
v
Ride Completed
|
v
View Ride History
```

7. Rider Flow



8. Admin Flow



9. Suggested Java Package Structure

```
src/
|
+-- main/
|   +-- Main.java
|
+-- users/
|   +-- User.java
|   +-- Customer.java
|   +-- Rider.java
|   +-- Admin.java
|   +-- Authenticatable.java
|
+-- package_management/
|   +-- City.java
|   +-- TourPackage.java
|   +-- Destination.java
|   +-- PackageSection.java
|
+-- vehicle/
|   +-- Vehicle.java
|   +-- Bike.java
|   +-- Auto.java
|   +-- Car.java
|
+-- booking/
|   +-- Booking.java
|
+-- ride/
|   +-- Ride.java
|
+-- pricing/
|   +-- PricingCalculator.java
|
+-- earning/
|   +-- Earning.java
|
+-- exceptions/
|   +-- InvalidLoginException.java
|   +-- InvalidBookingException.java
|   +-- PackageNotFoundException.java
|   +-- RiderNotAvailableException.java
|   +-- InvalidRideStateException.java
|   +-- InvalidInputException.java
|
+-- data/
|   +-- SystemData.java
|   +-- SystemConstants.java
```

10. Main Console Menu

```
=====
CITY TOUR PACKAGE SYSTEM
=====

1. Customer
2. Rider
3. Admin
4. Register
5. Exit

Enter your choice:
```

11. Customer Menu

```
1. My Profile
2. View Cities
3. View Tour Packages
4. Book Tour Package
5. My Booking
6. Current Ride
7. Ride History
8. Cancel Booking
9. Logout
```

12. Rider Menu

1. My Profile
2. Assigned Rides
3. Accept Ride
4. Reject Ride
5. Mark Reached
6. Mark Picked Up
7. Start Ride
8. Complete Package Section
9. Complete Ride
10. My Earnings
11. Ride History
12. Logout

13. Admin Menu

1. Manage Customers
2. Manage Riders
3. Manage Cities
4. Manage Tour Packages
5. Manage Destinations
6. Manage Vehicles
7. Configure Pricing
8. Assign Rider
9. Manage Bookings
10. Monitor Rides
11. View Reports
12. Logout

14. Exception Handling

Exception	When Used
InvalidLoginException	Wrong username/password or invalid login.
InvalidBookingException	Invalid number of persons, inactive package or invalid vehicle.
PackageNotFoundException	Selected package does not exist.
RiderNotAvailableException	No available rider can be assigned.
InvalidRideStateException	Rider attempts an invalid ride operation.
InvalidInputException	Invalid console input or invalid user choice.
InputMismatchException	Wrong input type entered in Scanner.
NumberFormatException	Invalid number conversion.

15. Dynamic Pricing

Final Price
=
Base Package Price
+
Passenger Charge
+
Vehicle Charge
+
Additional Charges

Example:

Base Package Price	= Rs. 1000
Two Persons Charge	= Rs. 200
Car Charge	= Rs. 300

Final Price	= Rs. 1500

16. Final OOP Summary

- Abstraction → User and Vehicle
- Inheritance → Customer, Rider, Admin, Bike, Auto, Car
- Interface → Authenticatable
- Encapsulation → Private fields with controlled methods
- Polymorphism → Different vehicle pricing calculations
- Association → Customer–Booking, Booking–Rider, Ride–Earning

- Aggregation → City–TourPackage
- Composition → TourPackage–PackageSection
- Static → Shared ID counters and String status constants
- Non-static → Individual customer, booking, rider and ride data
- Arrays → Temporary in-memory data storage
- Date/Time → Booking, pickup, section completion, ride completion
- Exception Handling → Custom and built-in exceptions